

PROGRAMACIÓN AVANZADA · UNIDAD 1

Cazar el cuello de botella

Medir, encontrar dónde se va el tiempo
y optimizar sin romper nada.

REPASO

Tres preguntas para cualquier problema

1

¿Qué estructura?

Pila, cola, lista o diccionario: depende de cómo vas a acceder a los datos.

2

¿Cómo sabes que está bien?

Casos de prueba con la salida esperada escrita antes, y un verificador.

3

¿Cuánto cuesta?

Medir con distintos tamaños y contar operaciones.

REPASO

Lo que importa es cómo crece

Si duplicas los datos...	Crece	Ejemplos
el costo no cambia	constante	isset en un diccionario, array_pop
el costo se duplica	lineal	in_array, un foreach
el costo se cuadruplica	cuadrático	array_shift en un ciclo, un ciclo dentro de otro

Un algoritmo cuadrático puede ir perfecto con 100 datos y ser inusable con 100.000.

Un cuello de botella es la parte del código donde se va casi todo el tiempo.

Casi siempre: un ciclo dentro de otro ciclo.

EFICIENCIA

Los ciclos que no se ven

Función	Qué hace por dentro
<code>in_array(\$x, \$lista)</code>	recorre la lista hasta encontrar \$x
<code>array_search(\$x, \$lista)</code>	lo mismo, y devuelve la clave
<code>array_column(\$filas, 'rut')</code>	recorre todas las filas para armar una lista nueva
<code>array_shift(\$cola)</code>	saca el primero y renumera todo el array

Una de estas **dentro de un foreach** es un ciclo dentro de otro, aunque solo veas uno.

EFICIENCIA

El método, en seis pasos

01

Verifica que funciona

Casos de prueba antes de tocar nada.

02

Mide con tres tamaños

Duplicando: 1.000, 2.000, 4.000.

03

Cuenta operaciones

Un contador da lo mismo en cualquier máquina.

04

Busca el ciclo dentro del ciclo

Incluidos los escondidos.

05

Optimiza

Cambia la estructura o saca trabajo del ciclo.

06

Verifica y mide otra vez

La versión nueva debe dar lo mismo que la original.

DEMO

Minimarket: total por cliente registrado

Entrada

Clientes registrados (rut, nombre) y pedidos (rut, monto).

Proceso

Sumar los montos de cada RUT registrado. Los RUT no registrados se ignoran.

Salida

Un diccionario
rut => total.

DEMO

Versión 1: funciona

```
<?php
declare(strict_types=1);

function totalRegistradosLenta(array $clientes, array $pedidos): array
{
    $totales = [];
    foreach ($pedidos as $p) {
        if (in_array($p['rut'], array_column($clientes, 'rut'), true)) {
            $totales[$p['rut']] = ($totales[$p['rut']] ?? 0) + $p['monto'];
        }
    }
    return $totales;
}
```

DEMO

Datos de prueba y medición

```
function generar(int $n): array
{
    $clientes = $pedidos = [];
    for ($i = 1; $i ≤ $n; $i++) {
        $clientes[] = ['rut' ⇒ "{$i}-K", 'nombre' ⇒ "Cliente {$i}"];
        $pedidos[] = ['rut' ⇒ rand(1, (int) ($n * 1.2)) . '-K', 'monto' ⇒ rand(500, 20000)];
    }
    return [$clientes, $pedidos];
}

foreach ([1000, 2000, 4000] as $n) {
    [$c, $p] = generar($n);
    $t0 = hrtime(true);
    totalRegistradosLenta($c, $p);
    printf("n=%5d  lenta: %8.2f ms\n", $n, (hrtime(true) - $t0) / 1e6);
}
```

DEMO

Antes y después

Datos	Versión 1 (ms)	Versión 2 (ms)
1.000		
2.000		
4.000		

¿Cuánto creció cada versión al duplicar los datos?

Break

A la vuelta: el caso de la ferretería.

DEMO

Versión 2: el mismo resultado, otra estructura

```
function totalRegistradosRapida(array $clientes, array $pedidos): array
{
    $registrados = array_flip(array_column($clientes, 'rut')); // una sola vez
    $totales = [];
    foreach ($pedidos as $p) {
        if (isset($registrados[$p['rut']])) {
            $totales[$p['rut']] = ($totales[$p['rut']] ?? 0) + $p['monto'];
        }
    }
    return $totales;
}
```

DEMO

Antes de celebrar: ¿dan lo mismo?

```
function verificar(string $caso, mixed $esperado, mixed $obtenido): void
{
    printf("[%s] %s\n", $esperado ≡ $obtenido ? ' OK ' : 'FALLA', $caso);
}
```

```
[$c, $p] = generar(500);
verificar('lenta y rápida dan lo mismo',
    totalRegistradosLenta($c, $p), totalRegistradosRapida($c, $p));
```

TALLER

Sospechoso 1: asistentes inscritos

Mide · encuentra el ciclo escondido · optimiza · verifica

```
<?php
declare(strict_types=1);

$n = 4_000; // prueba con 1_000, 2_000 y 4_000
$inscritos = array_map(fn($i) => "rut-{$i}", range(1, $n));
$asistentes = array_map(fn($i) => 'rut-' . rand(1, 2 * $n), range(1, $n));

$validos = 0;
foreach ($asistentes as $rut) {
    if (in_array($rut, $inscritos, true)) {
        $validos++;
    }
}

echo "Asistentes inscritos: {$validos}", PHP_EOL;
```

TALLER

Sospechoso 2: la fila de atención

Mide · encuentra el ciclo escondido · optimiza · verifica

```
<?php
declare(strict_types=1);

$fila = range(1, 20_000); // prueba con 10_000, 20_000 y 40_000

$atendidos = 0;
while ($fila) {
    $cliente = array_shift($fila);
    $atendidos++;
}

echo "Atendidos: {$atendidos}", PHP_EOL;
```

TALLER

Sospechoso 3: RUT repetidos

Mide · encuentra el ciclo escondido · optimiza · verifica

```
<?php
declare(strict_types=1);

$n = 4_000; // prueba con 1_000, 2_000 y 4_000
$ruts = array_map(fn($i) => 'rut-' . rand(1, 2 * $n), range(1, $n));

$duplicados = [];
foreach ($ruts as $i => $a) {
    foreach ($ruts as $j => $b) {
        if ($i < $j && $a === $b) {
            $duplicados[$a] = true;
        }
    }
}

echo count($duplicados), " RUT repetidos", PHP_EOL;
```

ENSAYO

Ferretería: un caso completo

A

Elegir estructuras

Tres necesidades, una estructura para cada una, fundamentada.

B

Diseñar e implementar

Pseudocódigo, PHP y casos de prueba con verificador.

C

Mejorar la eficiencia

Medir, encontrar el cuello de botella, optimizar y verificar.

Un caso completo: elegir estructuras, diseñar, probar y optimizar.

Elige una estructura para cada necesidad

a

Despachos

Pedidos de despacho que se atienden por orden de llegada.

b

Precios

Consultar el precio de un producto por su código.

c

Deshacer

El botón «deshacer» del carrito de venta.

Para cada una: ¿qué estructura?, ¿por qué? (acceso, inserción y eliminación), ¿qué pasaría con una estructura inadecuada?

Monto vendido por categoría

Entrada

Un catálogo (código => nombre, categoría, precio) y una lista de ventas (código, cantidad).

Salida

Un diccionario
categoría => monto.

Regla

Una venta con un código que no está en el catálogo se ignora.

Entrega: pseudocódigo en papel · PHP · tabla con 5 casos de prueba (vacío, uno solo, repetidos, límite, inválido) · verificador

Los datos

```
<?php
declare(strict_types=1);

$catalogo = [
    'MAR-01' => ['nombre' => 'Martillo',      'categoria' => 'herramientas', 'precio' => 8990],
    'TOR-10' => ['nombre' => 'Tornillo 1"',  'categoria' => 'fijaciones',  'precio' => 50],
    'LLA-22' => ['nombre' => 'Llave inglesa', 'categoria' => 'herramientas', 'precio' => 12490],
    'CLA-05' => ['nombre' => 'Clavo 2"',     'categoria' => 'fijaciones',  'precio' => 20],
    'PIN-03' => ['nombre' => 'Pintura blanca', 'categoria' => 'pinturas',    'precio' => 15990],
];

$ventas = [
    ['codigo' => 'MAR-01', 'cantidad' => 2],
    ['codigo' => 'TOR-10', 'cantidad' => 100],
    ['codigo' => 'MAR-01', 'cantidad' => 1],
    ['codigo' => 'XXX-99', 'cantidad' => 4],
    ['codigo' => 'CLA-05', 'cantidad' => 250],
];
```

ENSAYO · PARTE C

Funciona, pero es lento

Mídelo, encuentra el cuello de botella, optimízalo y demuestra que da lo mismo.

```
// Productos que no se vendieron ni una vez
function sinVentas(array $catalogo, array $ventas): array
{
    $resultado = [];
    foreach ($catalogo as $codigo => $producto) {
        $vendido = false;
        foreach ($ventas as $v) {
            if ($v['codigo'] === $codigo) {
                $vendido = true;
            }
        }
        if (!$vendido) {
            $resultado[] = $codigo;
        }
    }
    return $resultado;
}
```

DEFENSA

Prepárate para responder

1. ¿Por qué la estructura que elegiste es la más adecuada?
2. ¿Cómo comprobaste que el algoritmo funciona correctamente?
3. ¿Qué cambio tuvo mayor impacto en la eficiencia?
4. ¿Cómo cambiaría tu solución si aumentara el volumen de datos?